# 15. Form Handling in React

## ▼ What is `Form Handling` in React?

- **Form handling** in React means **managing the input fields** (like text, email, password, radio, select, etc) and their values when a user types something.
- **It includes:**
  - Capturing user inputs
  - Updating the state when input changes
  - Submitting the form
- **Forms allow users to:**
  - Enter text
  - Select options
  - Choose radio buttons
  - Tick checkboxes
  - Submit data to a backend

---

## ▼ Why Form Handling is Important?

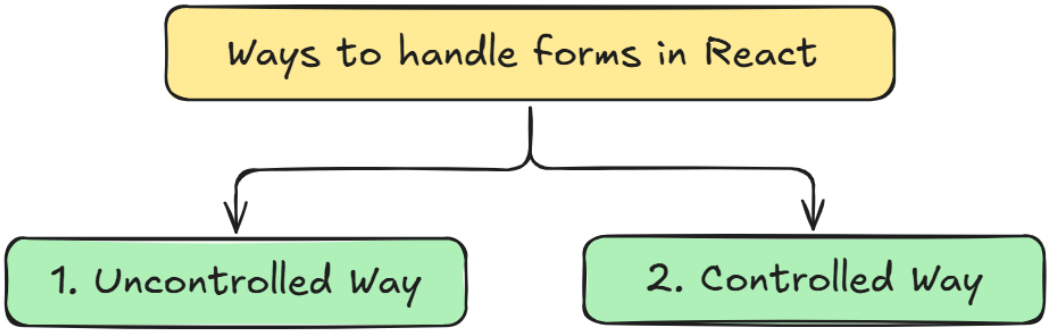
- **Without forms:**
  - No login page
  - No registration page
  - No contact form
  - No feedback system
  - No admin panel
- So **forms are the backbone of every real-world application.**

---

## ▼ Ways to Handle Forms in React

- **There are two main approaches:**

1. Uncontrolled Components
2. Controlled Components




---

# ▼ 1. Uncontrolled Components in React

- In uncontrolled components, the DOM controls the form data, not the React state.
- This type of component doesn't care about real time input changes.
- You can access the current value of the input using a ref after the form is submitted or whenever needed.
- `Ref` attribute is mandatory.
- **Why should we not use an uncontrolled way?**
- `For Example` → In the first approach, you let the bike take control. You sit on the bike and let it decide the direction and speed. You might try to make it go in a certain direction by leaning your body, but ultimately, the bike decides where to go.
- React does NOT track changes in real-time
- Less control over validation

## ▼ Flow of Uncontrolled Components

- Create a ref using `useRef` (In functional component)
- Attach the ref to the input field.
- On form submit, **read** `value` **from** `ref`.
- DOM handles the form field updates (Not React State)

---

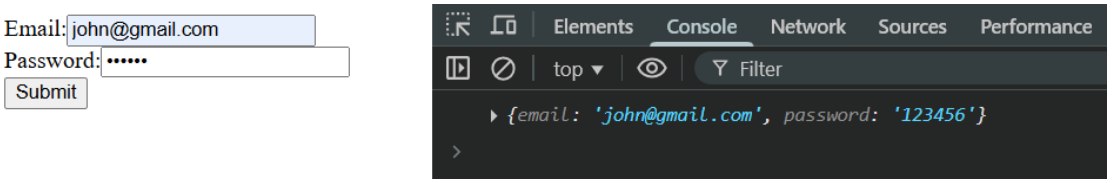
## ▼ Example: Uncontrolled Form using `useRef`

- **Code:**

```
import React, { useRef } from "react";

const UncontrolledForm = () => {
  let emailRef = useRef("");
  let passwordRef = useRef("");

  let handleSubmit = (e) => {
    e.preventDefault();
    let email = emailRef.current.value;
    let password = passwordRef.current.value;
    console.log({ email, password });
  };
  return (
    <>
      <form>
        <div>
          <label htmlFor="email">Email:</label>
          <input type="email" id="email" name="email" ref={emailRef} /
>
        </div>
        <div>
          <label htmlFor="password">Password:</label>
          <input
            type="password"
            id="password"
            name="password"
            ref={passwordRef}
          />
        </div>
        <div>
          <button onClick={handleSubmit}>Submit</button>
        </div>
      </form>
    </>
  );
```

```
};

export default UncontrolledForm;
```

- **Output:**




# ▼ 2. Controlled Components in React

- In this approach, the form data is handled by React through hooks such as the `useState` hook.
- The component decides when and how to update the state, and it re-renders itself based on the state changes.
- `For Example` → In the second approach, you take control of the bike. You hold the handlebars and pedal, and you decide where to go and how fast to ride. You can easily slow down or speed up as needed.

## ▼ Flow of Controlled Components

1. Declare state variables to hold form data.
2. Use `onChange` handler to update state on every keystroke.
3. Bind the input `value` to the corresponding state.
4. `onSubmit`, use the state values to perform operations.

## ▼ 4 Key Things Required

1. `state (object)` : Stores the  input value
2. `onChange` **handler:** Updates the state
3. `value` **attribute:** Connects state to input
4. `name` **attribute:** idntifies field

## ▼ How to get the Data

1. Inside the event object we have a property called `target` to get our targeted element.

   **Syntax**: **e.target**

2. To get the value, we have to use the value property on the targeted element (present in input elements only)

   **Syntax**: **e.target.value**

3. To get the name of our input element, we have to use the name property present on the targeted element.

   **Syntax: e.target.name**

4. Inside the `onChange` event, update the state of that input element by getting the value that the user has entered.

5. Don't add a click event on your button, instead add a submit event on your form and then you can get your data using the state's which you have created for each respective element.

   **Syntax: onSubmit**

---

# ▼ Handling Form Submission

- **Always prevent default form submission behavior:**
- **Code:**

```
const handleSubmit = (e) => {
  e.preventDefault();
  // Logic here
};
```

---

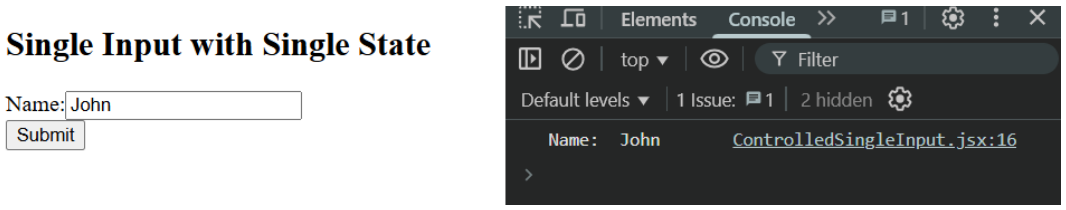
# ▼ Example 1: Single Input with Single State

- **Code:**

```
import React, { useState } from "react";

const ControlledSingleInput = () => {
```

```
  let [name, setName] = useState("");

  //Event Handler for change
  let handleChange = (e) => {
    let name = e.target.name;
    let value = e.target.value;
    setName(value);
  };

  //Event Handler for submit
  let handleSubmit = (e) => {
    e.preventDefault();
    console.log("Name: ", name);
  };

  return (
    <>
      <h2>Single Input with Single State</h2>
      <form onSubmit={handleSubmit}>
        <div>
          <label htmlFor="name">Name:</label>
          <input
            type="text"
            name="name"
            id="name"
            value={name}
            onChange={handleChange}
          />
        </div>
        <div>
          <button>Submit</button>
        </div>
      </form>
    </>
  );
};

export default ControlledSingleInput;
```

- **Output:**




---

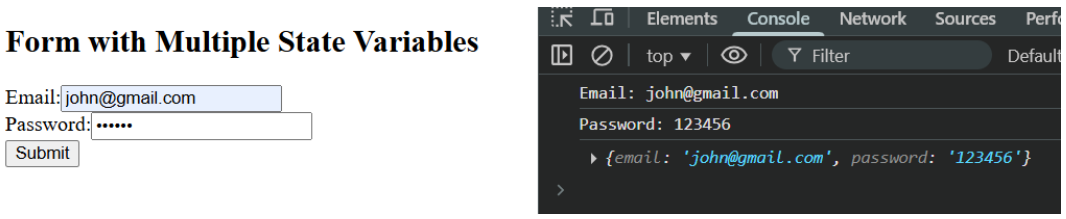
# ▼ Example 2: Multiple Inputs with Multiple State Variables

- **Code:**

```
import React, { useState } from "react";

const ControlledMultiState = () => {
  // Separate states for each field
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");

  const handleEmailChange = (e) => setEmail(e.target.value);
  const handlePasswordChange = (e) => setPassword(e.target.value);

  const handleSubmit = (e) => {
    e.preventDefault();
    // console.log("Email:", email);
    // console.log("Password:", password);
    console.log({ email, password });
  };

  return (
    <>
      <h2>Form with Multiple State Variables</h2>
      <form onSubmit={handleSubmit}>
        <div>
          <label>Email:</label>
          <input
            type="email"
            value={email}
```

```
          onChange={handleEmailChange}
          name="email"
        />
      </div>
      <div>
        <label>Password:</label>
        <input
          type="password"
          value={password}
          onChange={handlePasswordChange}
          name="password"
        />
      </div>
      <button type="submit">Submit</button>
    </form>
  </>
);
};

export default ControlledMultiState;
```

- **Output:**




# ▼ Example 3: Single State Object to Handle Multiple Inputs

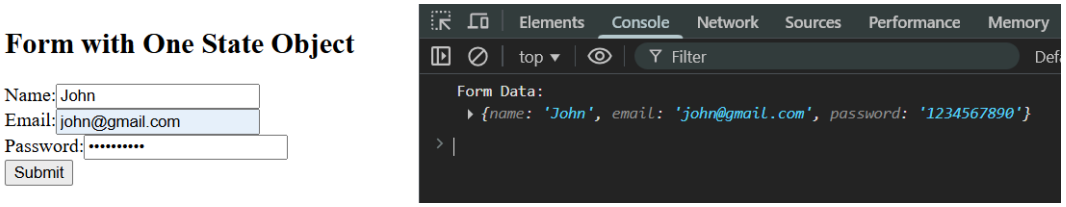
- **Code:**

```
import React, { useState } from "react";

const ControlledSingleObject = () => {
  //single state object
```

```
const [formData, setFormData] = useState({
  name: "",
  email: "",
  password: "",
});

// Dynamic field update
const handleChange = (e) => {
  const { name, value } = e.target;
  setFormData({ ...formData, [name]: value });
};

const handleSubmit = (e) => {
  e.preventDefault();
  console.log("Form Data:", formData);
};

return (
  <>
    <h2>Form with One State Object</h2>
    <form onSubmit={handleSubmit}>
      <div>
        <label>Name:</label>
        <input
          type="text"
          name="name"
          value={formData.name}
          onChange={handleChange}
        />
      </div>
      <div>
        <label>Email:</label>
        <input
          type="email"
          name="email"
          value={formData.email}
          onChange={handleChange}
        />
```

```
      </div>
      <div>
        <label>Password:</label>
        <input
          type="password"
          name="password"
          value={formData.password}
          onChange={handleChange}
        />
      </div>
      <button type="submit">Submit</button>
    </form>
  </>
);
};

export default ControlledSingleObject;
```

- **Output:**



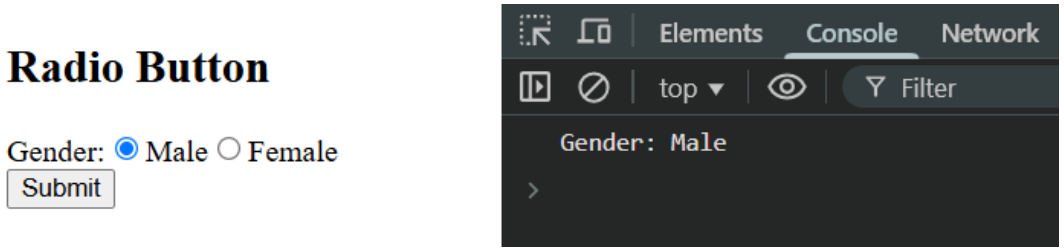

# ▼ Handling Radio Buttons

- **Code:**

```
import React, { useState } from "react";

const RadioInput = () => {
  let [gender, setGender] = useState("");

  let handleChange = (e) => {
    setGender(e.target.value);
  };
```

```
  let handleSubmit = (e) => {
    e.preventDefault();
    console.log("Gender:", gender);
  };

  return (
    <>
      <h2>Radio Button</h2>
      <form onSubmit={handleSubmit}>
        <div>
          <label>Gender:</label>
          <input
            type="radio"
            name="gender"
            value="Male"
            onChange={handleChange}
          />
          Male
          <input
            type="radio"
            name="gender"
            value="Female"
            onChange={handleChange}
          />
          Female
        </div>
        <button type="submit">Submit</button>
      </form>
    </>
  );
};

export default RadioInput;
```

- **Output:**




# ▼ Handling Checkboxes

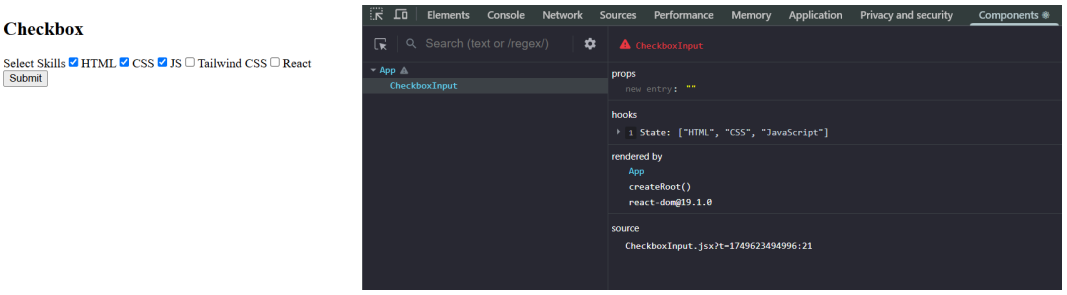
- **Code:**

```
import React, { useState } from "react";

const CheckboxInput = () => {
  let [skills, setSkills] = useState([]);

  let handleChange = (e) => {
    let { value, checked } = e.target;
    //if it is checked it will add the skill
    if (checked) {
      //add the previous checked values also
      setSkills((prevSkills) => {
        return [...prevSkills, value];
      });
    } else {
      //remove skill from the array if it is unchecked
      setSkills((prevSkills) => prevSkills.filter((skill) => skill !== value));
    }
  };

  let handleSubmit = (e) => {
    e.preventDefault();
    console.log("Skills: ", skills);
  };

  return (
    <>
      <h2>Checkbox</h2>
      <form onSubmit={handleSubmit}>
```

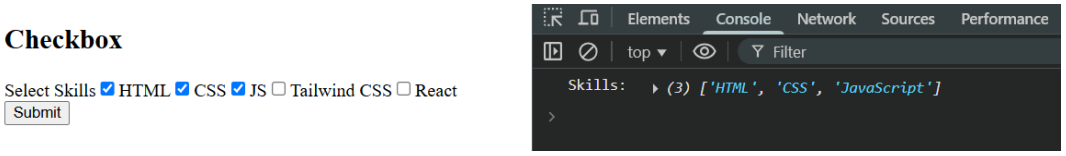
```
<div>
  <label>Select Skills</label>
  <label>
    <input
      type="checkbox"
      name="skills"
      id="skills"
      value="HTML"
      onChange={handleChange}
    />
    HTML
  </label>
  <label>
    <input
      type="checkbox"
      name="skills"
      id="skills"
      value="CSS"
      onChange={handleChange}
    />
    CSS
  </label>
  <label>
    <input
      type="checkbox"
      name="skills"
      id="skills"
      value="JavaScript"
      onChange={handleChange}
    />
    JS
  </label>
  <label>
    <input
      type="checkbox"
      name="skills"
      id="skills"
      value="Tailwind CSS"
```

```
            onChange={handleChange}
          />
          Tailwind CSS
        </label>
        <label>
          <input
            type="checkbox"
            name="skills"
            id="skills"
            value="React"
            onChange={handleChange}
          />
          React
        </label>
      </div>
      <button type="submit">Submit</button>
    </form>
  </>
);
};

export default CheckboxInput;
```

- **Output: Yes I am getting the array of skills but the problem is that when I deselect it not removing from the skills array.**




- **For that, we have to check if the value is already checked and the value you want to remove by unchecking the box.**




# ▼ Handling Select options

- **Code:**

```
import React, { useState } from "react";

const SelectInput = () => {
  let [course, setCourse] = useState("");

  let handleChange = (e) => {
    setCourse(e.target.value);
  };

  let handleSubmit = (e) => {
    e.preventDefault();
    console.log("Selected Course:", course);
  };

  return (
    <>
      <h2>Select Dropdown</h2>
      <form onSubmit={handleSubmit}>
        <div>
          <label>Select Course:</label>
          <select name="course" value={course} onChange={handleChang
e}>
            <option value="" disabled>
              --Select--
            </option>
            <option value="MERN">MERN Stack</option>
            <option value="MEAN">MEAN Stack</option>
            <option value="Java">Java Full Stack</option>
            <option value="Python">Python Full Stack</option>
          </select>
```

```
      </div>
      <button type="submit">Submit</button>
    </form>
  </>
 );
};

export default SelectInput;
```

- **Output:**

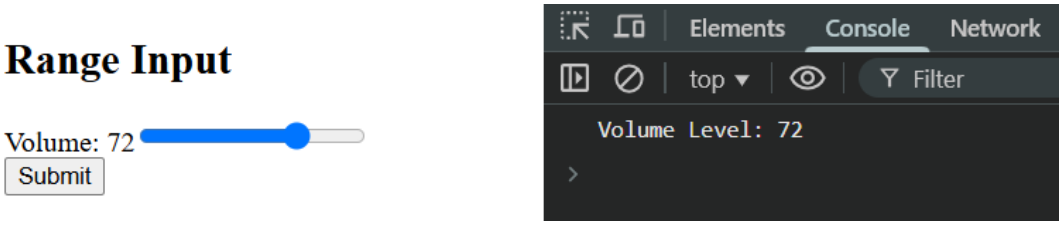
# ▼ Handling Range Input

- **Code:**

```
import React, { useState } from "react";

const RangeInput = () => {
 let [volume, setVolume] = useState(50);

 let handleChange = (e) => {
  setVolume(e.target.value);
 };

 let handleSubmit = (e) => {
  e.preventDefault();
  console.log("Volume Level:", volume);
 };

 return (
  <>
   <h2>Range Input</h2>
   <form onSubmit={handleSubmit}>
    <div>
```

```
        <label>Volume: {volume}</label>
        <input
          type="range"
          min="0"
          max="100"
          value={volume}
          onChange={handleChange}
        />
      </div>
      <button type="submit">Submit</button>
    </form>
  </>
 );
};

export default RangeInput;
```

- **Output:**



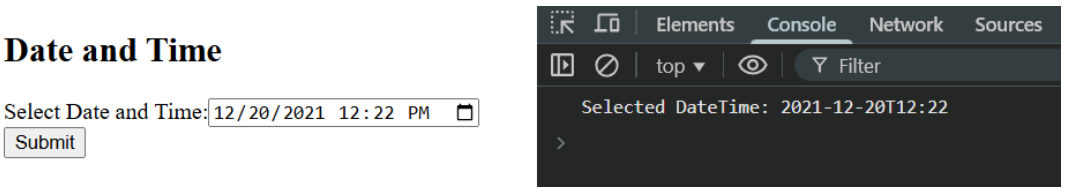

# ▼ Handling Date And Time

- **Code:**

```
import React, { useState } from "react";

const DateTimeInput = () => {
  let [dateTime, setDateTime] = useState("");

  let handleChange = (e) => {
    setDateTime(e.target.value);
  };

  let handleSubmit = (e) => {
```

```
    e.preventDefault();
    console.log("Selected DateTime:", dateTime);
  };

  return (
    <>
      <h2>Date and Time</h2>
      <form onSubmit={handleSubmit}>
        <div>
          <label>Select Date and Time:</label>
          <input
            type="datetime-local"
            name="dateTime"
            value={dateTime}
            onChange={handleChange}
          />
        </div>
        <button type="submit">Submit</button>
      </form>
    </>
  );
};

export default DateTimeInput;
```

- **Output:**




# ▼ Third Party Libraries

## ▼ 1. react-icons library

- `react-icons` is a popular **React library** that lets you easily use **icons from famous icon packs.**
- Command to install react-icons:

💡 npm install react-icons

---

## ▼ 2. react-hot-toast library

- `react-hot-toast` is a lightweight and customizable library for displaying toast notifications in React applications.
- It provides a simple way to add user-friendly notifications for various actions, such as success, error, or loading states.
- **Command to install react-hot-toast:**

💡 npm install react-hot-toast

- Import and place the `<Toaster />` component: Add the component at the top level of your application.

---